Write lean easy-to-maintain CSS

There are many ways to lean and simplify SCSS. The first step is to establish if custom code is needed at all.

Odoo's webclient has been designed to be modular, meaning that (potentially all) classes can be shared across views. Check the code before creating a new class. Chances are that there is already a class or an HTML tag doing exactly what you're looking for.

On top of that, Odoo relies on **Bootstrap** (BS), one of the most complete CSS frameworks available. The framework has been customized in order to match Odoo's design (both community and enterprise versions), meaning that you can use any BS class directly in Odoo and achieve a visual result that is consistent with our UI.

> **Warning**
>
> - The fact that a class achieves the desired visual result doesn't necessarily mean that it's the right one for the job. Be aware of classes triggering JS behaviors, for example.
> - Be careful about class semantics. Applying a **button class** to a **title** is not only semantically wrong, it may also lead to migration issues and visual inconsistencies.

The following sections describe tips to strip-down SCSS lines **when custom-code is the only way to go**.

Browser defaults

By default, each browser renders content using a *user agent stylesheet*. To overcome inconsistencies between browsers, some of these rules are overridden by **Bootstrap Reboot**.

At this stage all "browser-specific-decoration" rules have been stripped away, but a big chunk of rules defining basic layout information is maintained (or reinforced by *Reboot* for consistency reasons).

You can rely on these rules.

> **Example**

```
    display: block;
    /* not needed 99% of the time */
}
```

Example

In this instance, you may opt to switching the HTML tag rather than adding a new CSS rule.

```
span.element {
    display: block;
    /* replace <span> with <div> instead
       to get 'display: block' by default */
}
```

Here's a non-comprehensive list of default rules:

Tag / Attribute	Defaults
`<div/>` , `<section/>` , `<header/>` , `<footer/>` …	`display: block`
`<span/>` , `<a/>` , `<em/>` , `<b/>` …	`display: inline`
`<button/>` , `<label/>` , `<output/>` …	`display: inline-block`
`<img/>` , `<svg/>`	`vertical-align: middle`
`<summary/>` , `[role="button"]`	`cursor: pointer;`
`<q/>`	`:before {content: open-quote}` `:after {content: close-quote}`
…	…

See also

Bootstrap Reboot on GitHub

odoo docs

is to use a header tag (`<h1>` , `<h2>` , ...). Besides reboot rules, mostly all tags carry decorative styles defined by Odoo.

Example

Don't

XML SCSS

```
<span class="o_module_custom_title">
   Hello There!
</span>

<span class="o_module_custom_subtitle">
   I'm a subtitle.
</span>
```

Do

XML SCSS

```
<h5 class="o_module_custom_title">
   Hello There!
</h5>

<div class="o_module_custom_subtitle">
   <b><small>I'm a subtitle.</small></b>
</div>
```

Note

Besides reducing the amount of code, a modular-design approach (use classes, tags, mixins...) keeps the visual result consistent and easily **maintainable**.

odoo docs

Utility classes

Our framework defines a multitude of utility classes designed to cover almost all layout/design/interaction needs. The simple fact that a class already exists justifies its use over custom CSS whenever possible.

Take the example of `position-relative`.

```
position-relative {
   position: relative !important;
}
```

Since a utility-class is defined, any CSS line with the declaration `position: relative` is **potentially** redundant.

Odoo relies on the default **Bootstrap utility-classes** stack and defines its own using **Bootstrap API**.

> **See also**
>
> - **Bootstrap utility classes**
> - **Odoo custom utilities on github**

Handling utility-classes verbosity

The downside of utility-classes is the potential lack of readability.

> **Example**
>
> ```
> <myComponent t-attf-class="d-flex border px-lg-2 card
> {{props.readonly ? 'o_myComponent_disabled' : ''}}
> card d-lg-block position-absolute {{props.active ?
> 'o_myComponent_active' : ''}}  myComponent px-3"/>
> ```

To overcome the issue you may combine different approaches:

odoo docs

[odoo component] [bootstrap component] [css declaration order].

Example

```
<myComponent
   t-att-class="{
      o_myComponent_disabled: props.readonly,
      o_myComponent_active: props.active
   }"
   class="myComponent card position-absolute d-flex d-lg-block border px-3 px-l
/>
```

See also

Odoo CSS properties order

Get Help

Contact Support | Ask the Odoo Community